VISUAL NOTES • COMPLETE GUIDE

Visual Notes — RAG Decisions with Jev

Why I switched my RAG decisions to Jev...

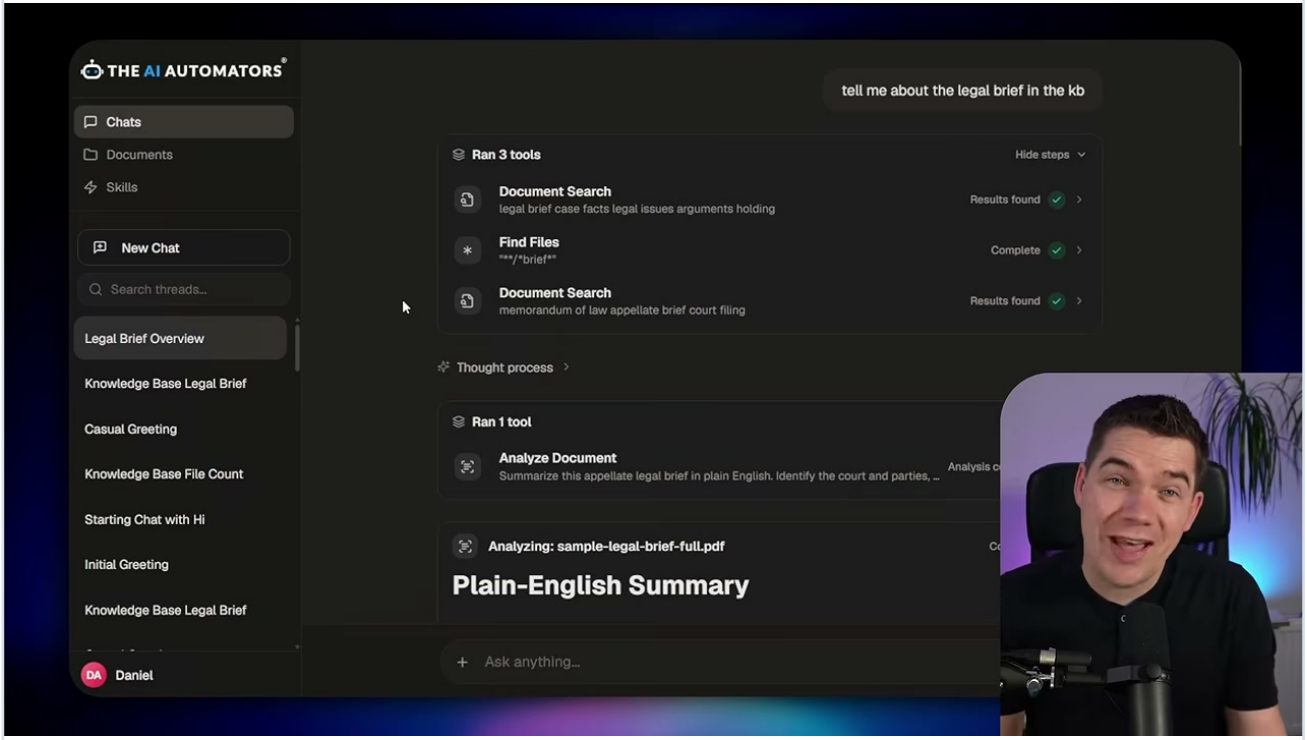
16:06

- 27 substantive visual(s), selected by content
- Repeated builds, presenter-only shots, and filler excluded
- Output length expands or contracts with the source material
- Claims, uncertainty, and caveats are labeled clearly

https://www.youtube.com/watch?v=olIve0D8pJQ

Jev as a Steerable RAG Reranker

A reranking instruction is applied to hybrid-search results before chunks reach the answering model.



NOTES

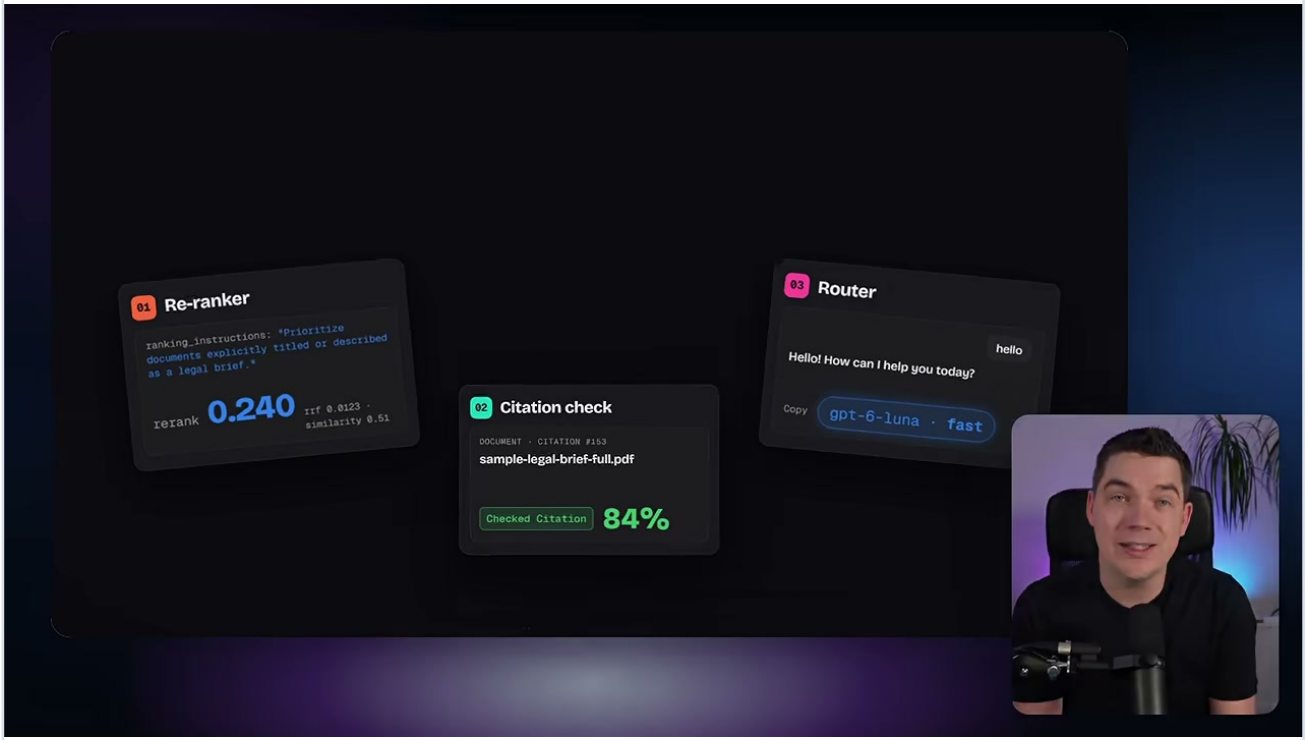
- The app retrieves candidate documents from a large knowledge base with hybrid search.
- Jev receives the retrieved documents, the user query, and a separate reranking instruction.
- Only chunks judged relevant are sent to the LLM; noisy chunks are discarded.

CORE IDEA

Reranking creates a decision layer between broad retrieval and expensive generation, reducing irrelevant context before the LLM answers.

Three Decision Jobs Inside the App

The demonstration uses Jev for reranking, citation checking, and model routing.



NOTES

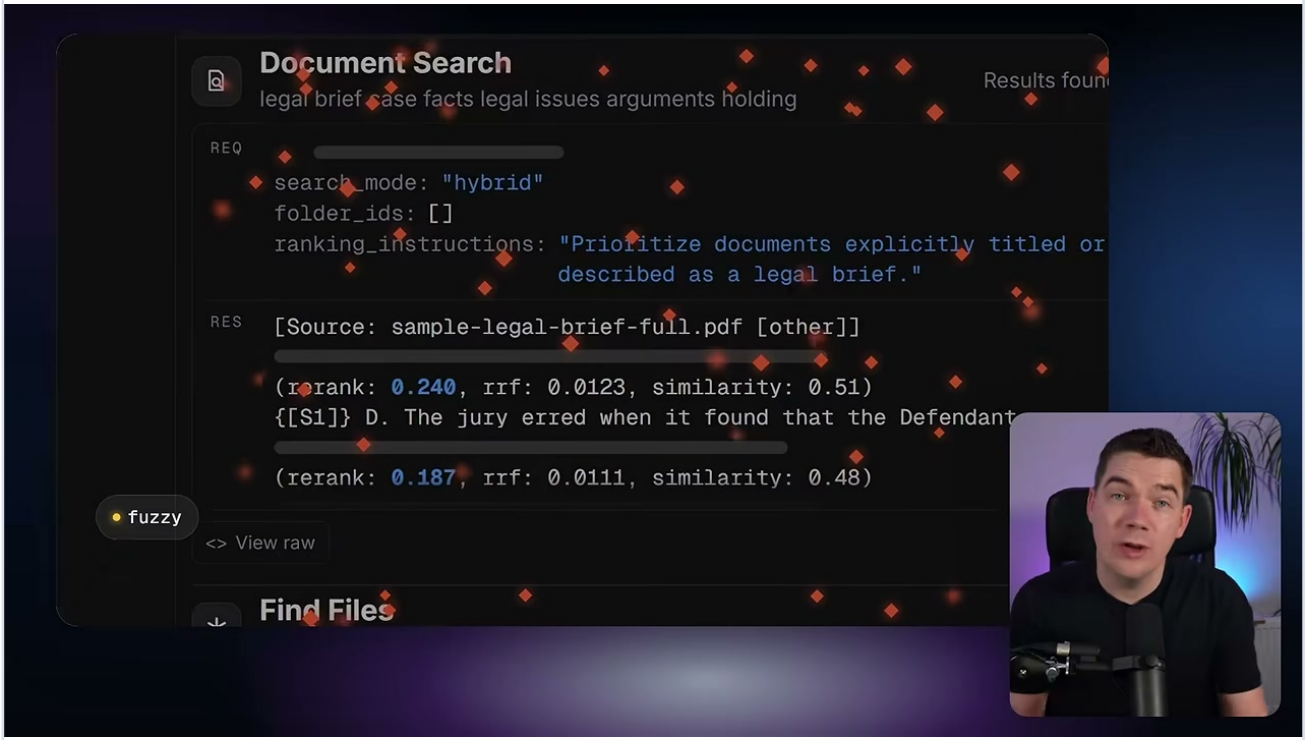
- Reranking prioritizes retrieved chunks against the query before generation.
- Citation checking compares generated claims with the passages cited as support.
- Model routing sends simple and complex requests to different models; the displayed speed and savings are demo claims, not independent benchmarks.

CORE IDEA

These examples frame decision models as small classifiers embedded at several control points rather than as replacements for the answering LLM.

Agentic Apps Contain Many Micro-Decisions

Fuzzy internal choices are separated from open-ended language generation.



NOTES

- An agent makes many small choices between a user's question and its final response.
- Some choices are fuzzy rather than deterministic, such as selecting a model or deciding which evidence is relevant.
- The proposed role for a System One model is to make those bounded decisions without generating prose.

CORE IDEA

Separating bounded decisions from open-ended generation makes each control step easier to specify and inspect.

Source: https://www.youtube.com/watch?v=ollve0D8pJQ

A Decision Model, Not a Chat Model

Jev is presented as a fast System One model that returns judgments instead of token-by-token prose.

NOTES

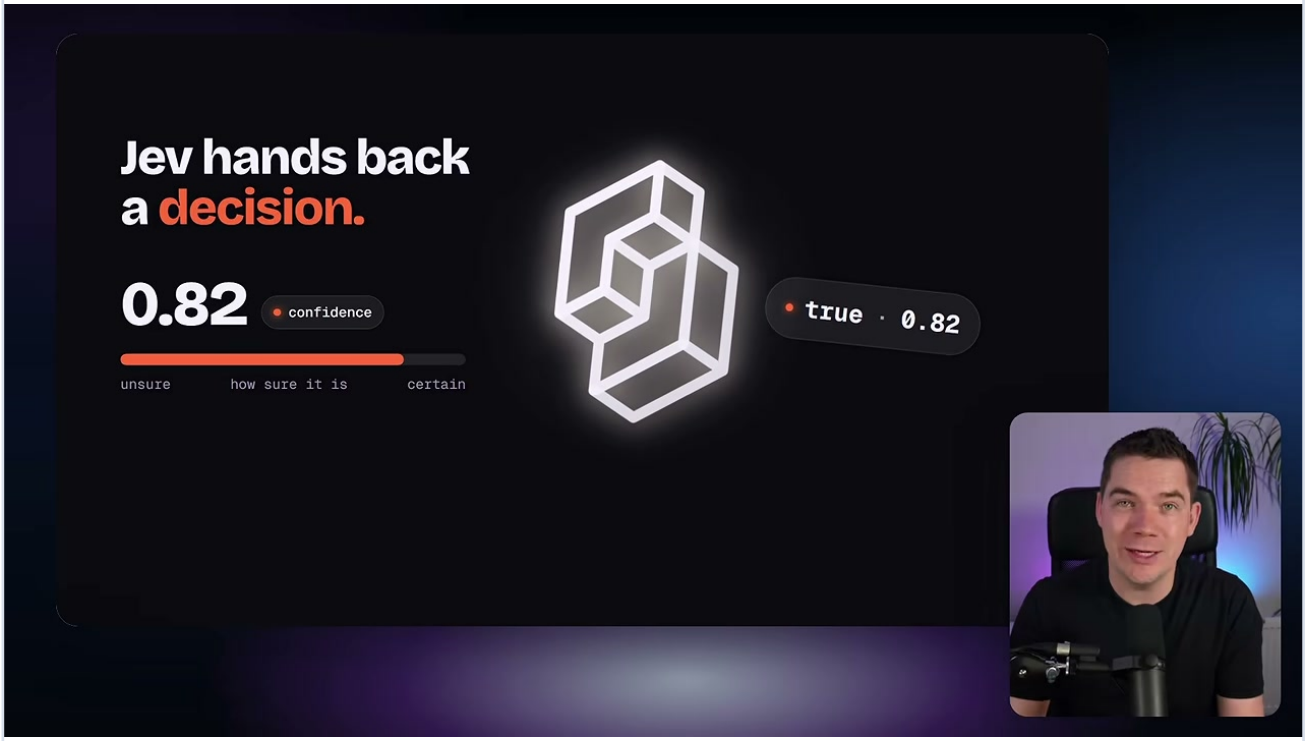
- Chat models answer by writing one token at a time, so their response visibly streams out.
- Jev is described as making a rapid, gut-feeling-style decision without an explicit reasoning trace.
- Its output is a decision plus numeric scores rather than free-form text.

CORE IDEA

The narrower output contract is suited to routing and filtering steps where an application needs a machine-readable choice, not an essay.

Decision Plus Confidence, With No Prose to Parse

Messy input is reduced to a schema-bound choice and score.



NOTES

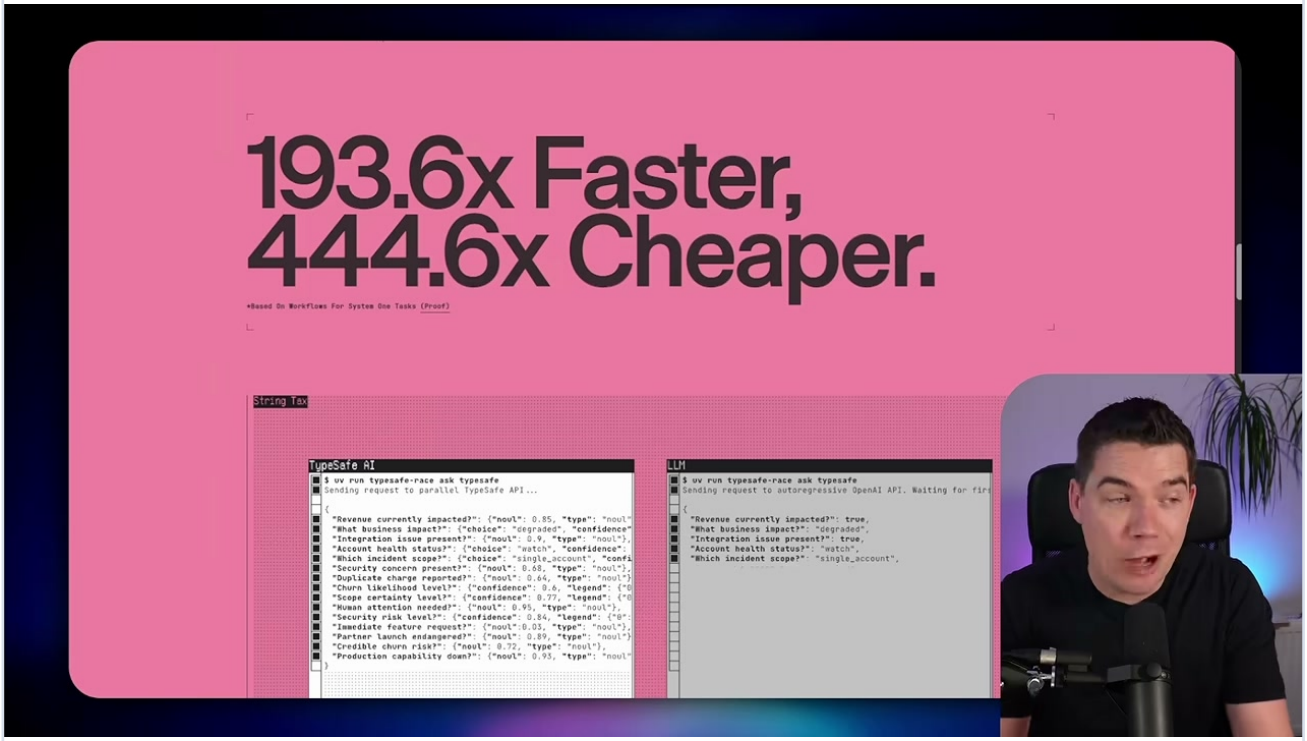
- The model accepts fuzzy text such as an email, passage, or conversation state.
- It returns a selected decision and a numeric value labeled as confidence.
- No independent evidence shows that these confidence values are calibrated probabilities.

CORE IDEA

A compact, structured result is simpler for application logic to consume, but score thresholds still require validation on the intended task.

Input-Heavy Pricing Model

The displayed pricing is just over four US cents per million input tokens, with free output.



NOTES

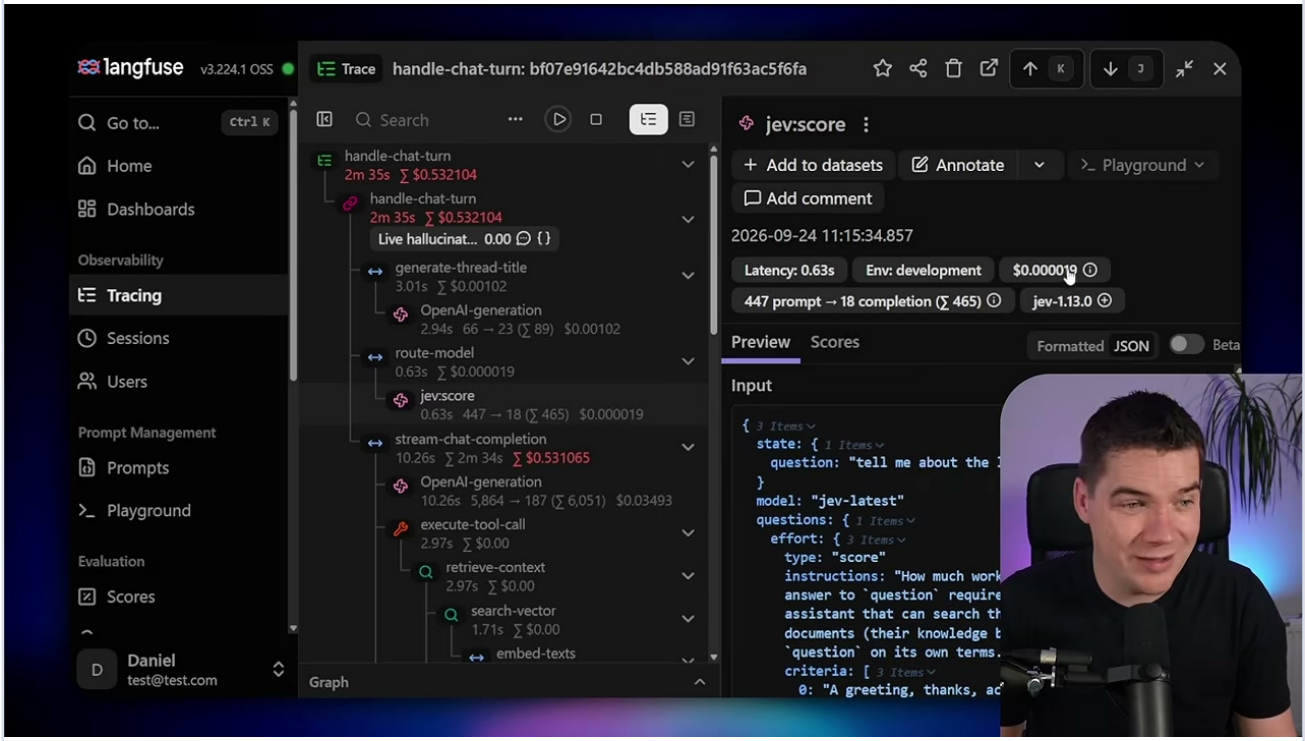
- The pricing explanation is tied to Jev returning only a few numbers rather than generated prose.
- The quoted rate is just over four US cents per million input tokens, with output described as free.
- Pricing and cost figures are demo claims, may change, and are not independently verified.

CORE IDEA

If the reported pricing holds for a deployment, high-volume classification calls may cost far less than using a generative model for each decision.

Reported Cost of the Demonstrated Calls

Routing, reranking, and citation checks are shown with small estimated costs.



NOTES

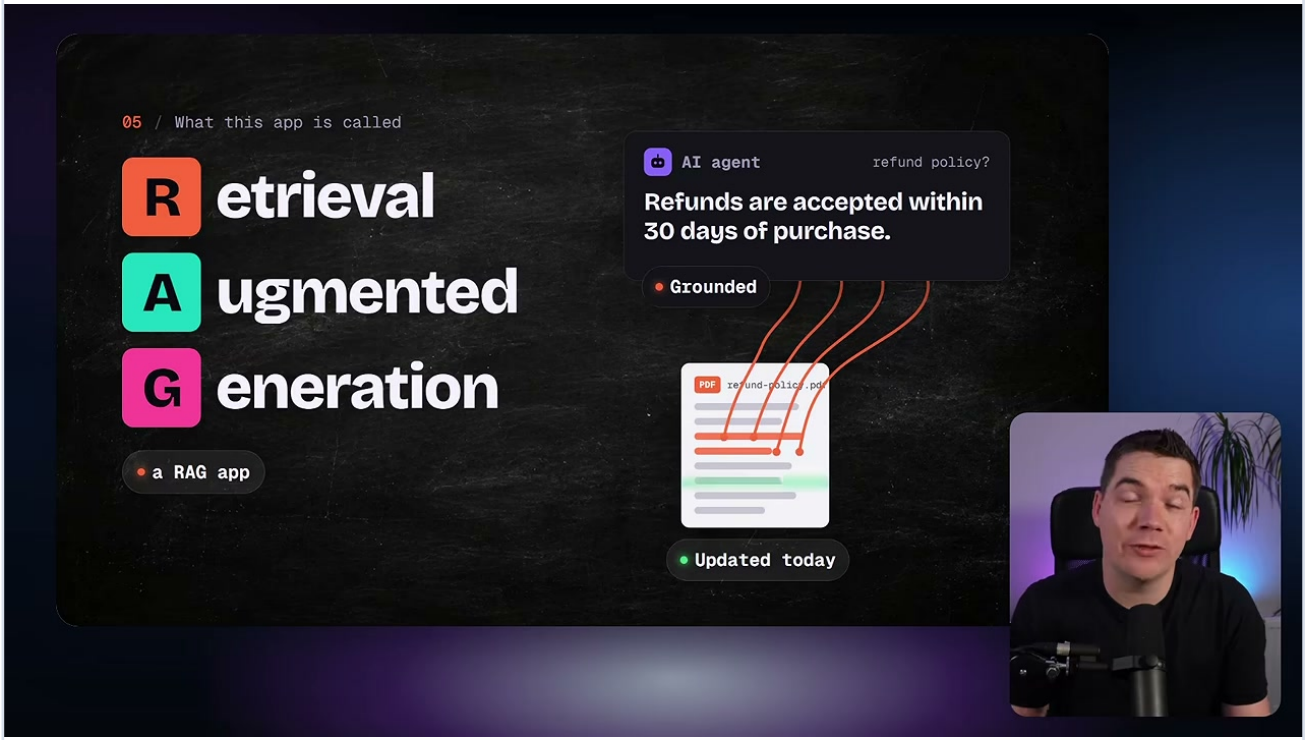
- The demonstrated routing call used just over 400 input tokens.
- The reranking example sent 60 chunks totaling almost 30,000 input tokens and was estimated at one-tenth of a US cent.
- The citation check used about 11,000 input tokens; token, cost, and speed figures come from the demo rather than an independent benchmark.

CORE IDEA

The economics are the core deployment argument, but teams should reproduce these estimates with their own prompts, traffic, and current pricing.

The Agentic RAG Loop

Documents are chunked at ingestion, searched at retrieval, and explored iteratively by an agent.



NOTES

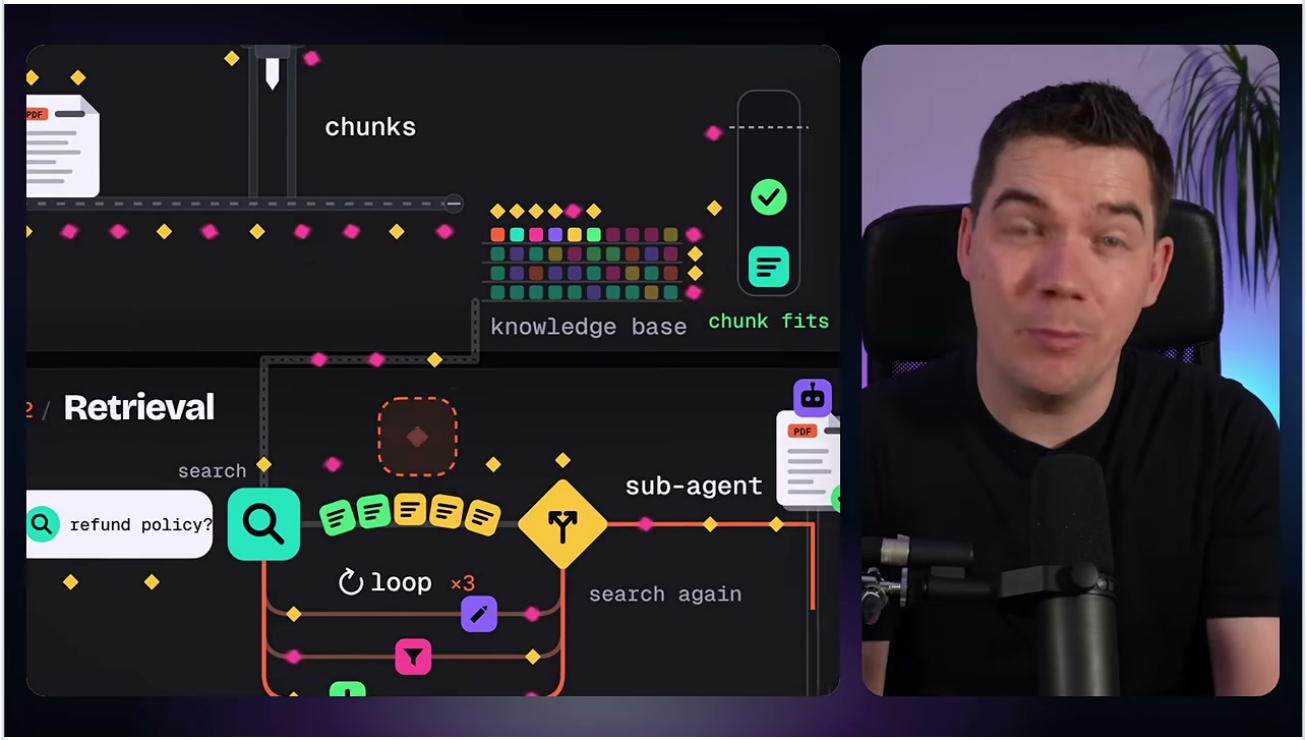
- Ingestion imports documents and divides them into smaller chunks for later retrieval.
- A retrieval pipeline searches the knowledge base for chunks similar to the agent's query.
- The agent can search again, change terms, or commission a sub-agent to inspect a full document.

CORE IDEA

Iteration makes the pipeline agentic, while also creating many intermediate points where relevance and routing decisions affect the final answer.

Micro-Decisions Span Ingestion and Retrieval

Fuzzy choices appear throughout both halves of a RAG pipeline.



NOTES

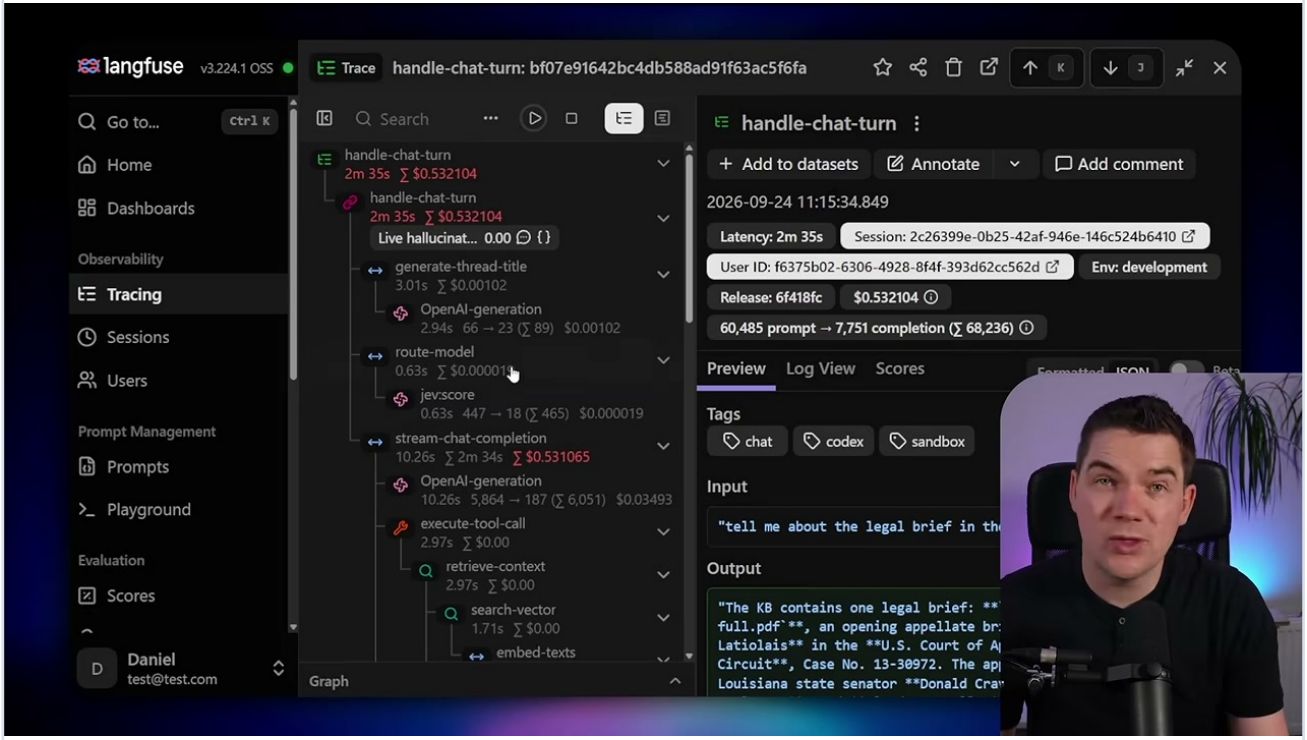
- Ingestion must decide how to classify, tag, and filter chunks before indexing.
- Retrieval must decide which search results are useful enough to keep exploring.
- Bounded, fuzzy choices are presented as a fit for a fast System One decision model.

CORE IDEA

Looking beyond the final answer exposes smaller decisions that can be isolated, measured, and improved independently.

A Langfuse Trace Exposes the Agent's Steps

The legal-brief request triggers searches, pattern matching, and a document-analysis sub-agent.



NOTES

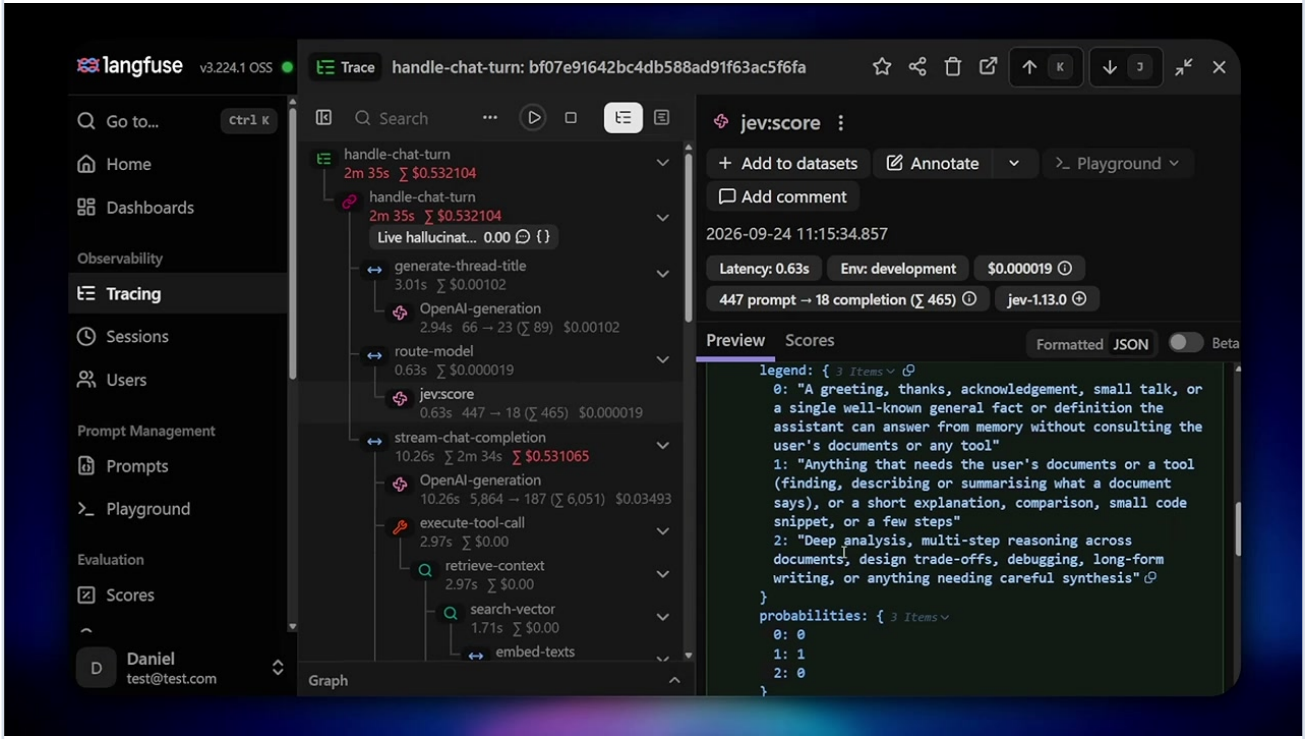
- The trace records hierarchical calls to both LLMs and decision models.
- For the legal-brief request, the agent performs document search, file pattern matching, and another search.
- It then provisions a sub-agent to analyze and summarize the document before producing a response.

CORE IDEA

Tracing turns an apparently single answer into an inspectable chain of decisions, making routing and retrieval behavior debuggable.

Routing by Required Work

The router scores options ranging from small talk to tool use and deep multi-hop analysis.



NOTES

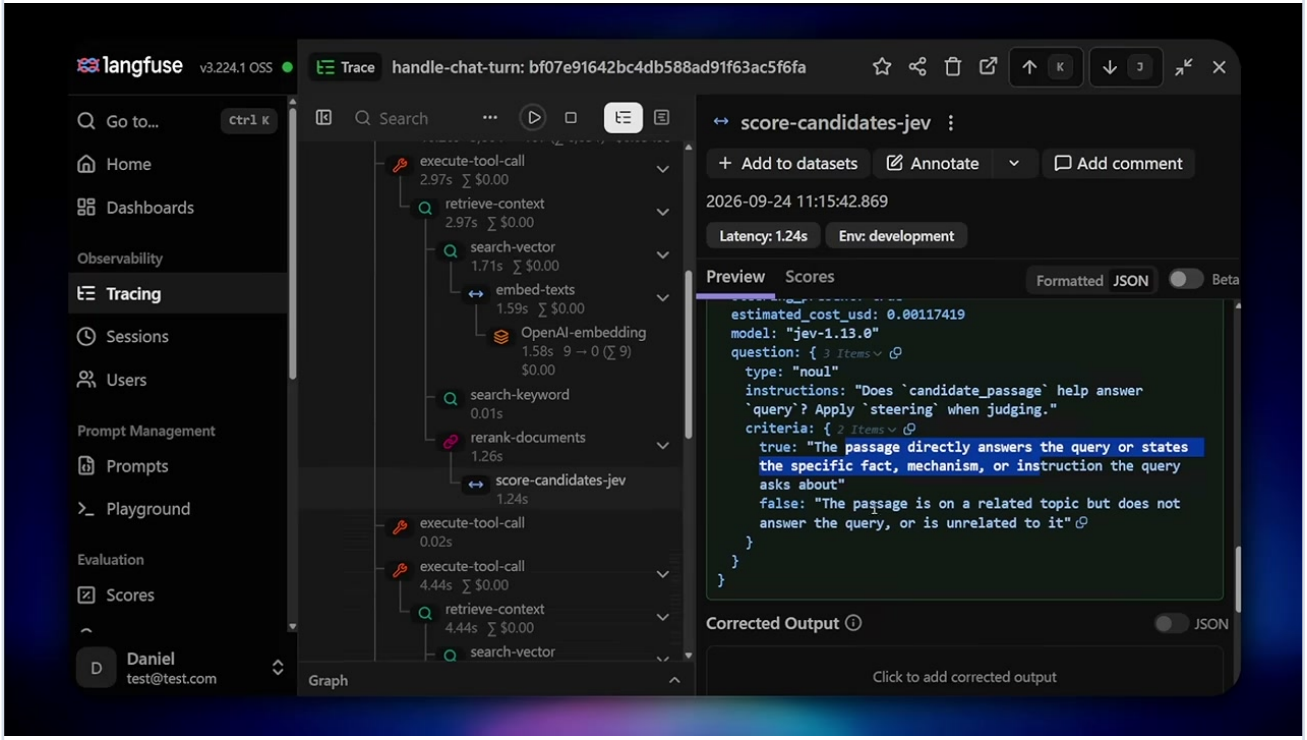
- The state includes the user's question, while instructions ask how much work a correct and complete answer requires.
- Criteria distinguish greetings, document or tool-dependent requests, and deep multi-step synthesis.
- Jev returns a number for each option; the shown route and apparent certainty are one demo result, not a routing benchmark.

CORE IDEA

Routing on explicit workload criteria can reserve expensive models for harder tasks, but the criteria and model mapping must be tested against real traffic.

Reranking Every Retrieved Chunk

The query and 60 candidates are scored on whether each passage helps answer the question.



NOTES

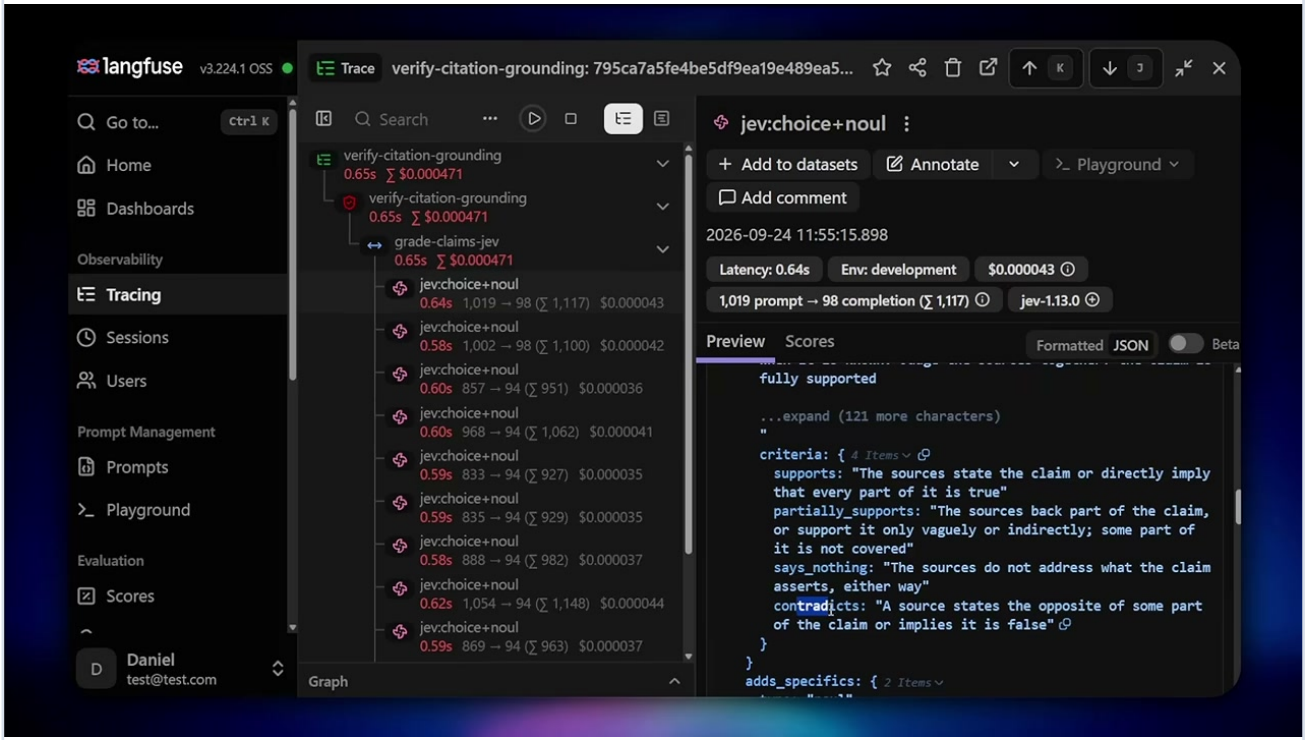
- The call passes the search query and the 60 most similar retrieved chunks to Jev.
- A yes-or-no instruction defines true as directly answering the query and false as related-but-unhelpful or unrelated.
- The 60-candidate count is an app example, and the returned confidence scores are not independently shown as calibrated.

CORE IDEA

Scoring candidates against the actual information need lets the agent continue with a smaller, more relevant evidence set.

Citation Support as a Four-Way Choice

A cited passage is classified as full, partial, absent, or contradictory support.



NOTES

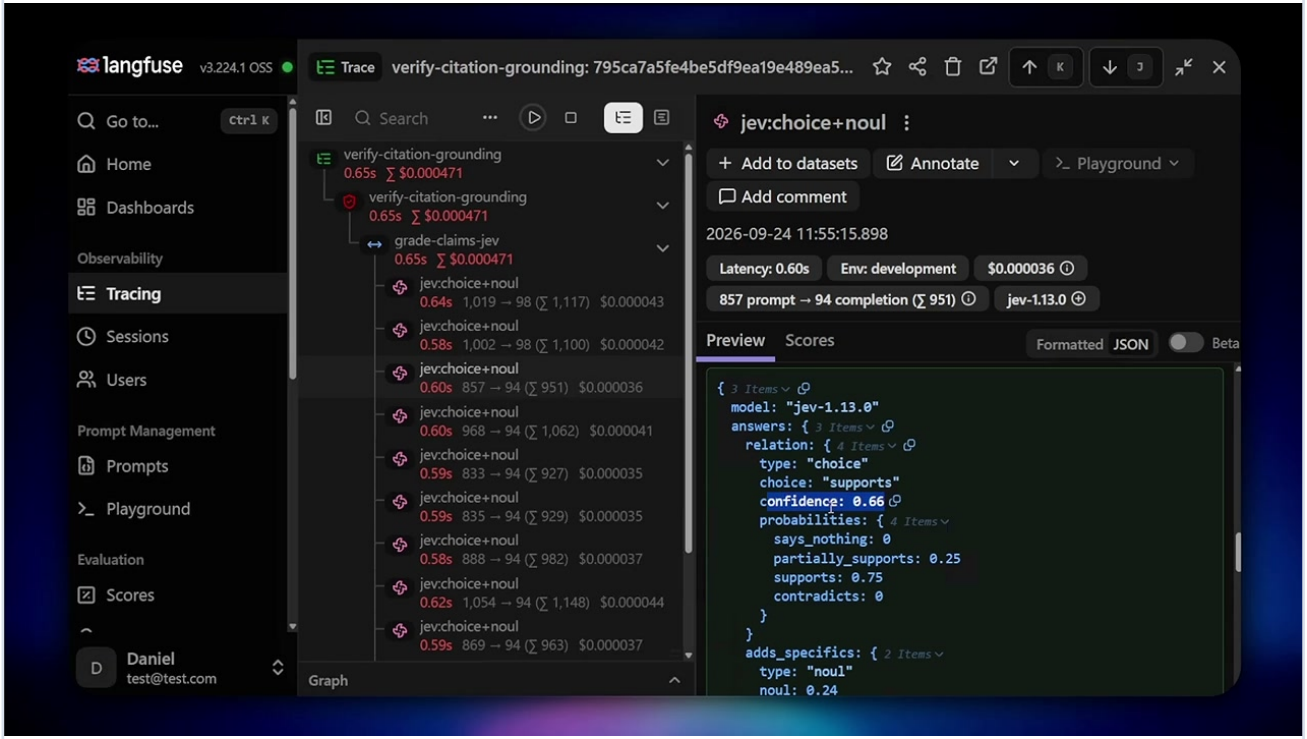
- Citation verification uses both a multiple-choice question and a yes-or-no question.
- The choice options distinguish fully supported, partially supported, unsupported, and contradicted claims.
- Displayed percentages are confidence outputs and are not independently demonstrated as calibrated probabilities.

CORE IDEA

A richer support label can identify partial evidence or contradiction instead of treating citation checking as a single pass/fail flag.

Scores Can Drive Application Logic

The demonstration proposes combining the selected label with a confidence threshold.



NOTES

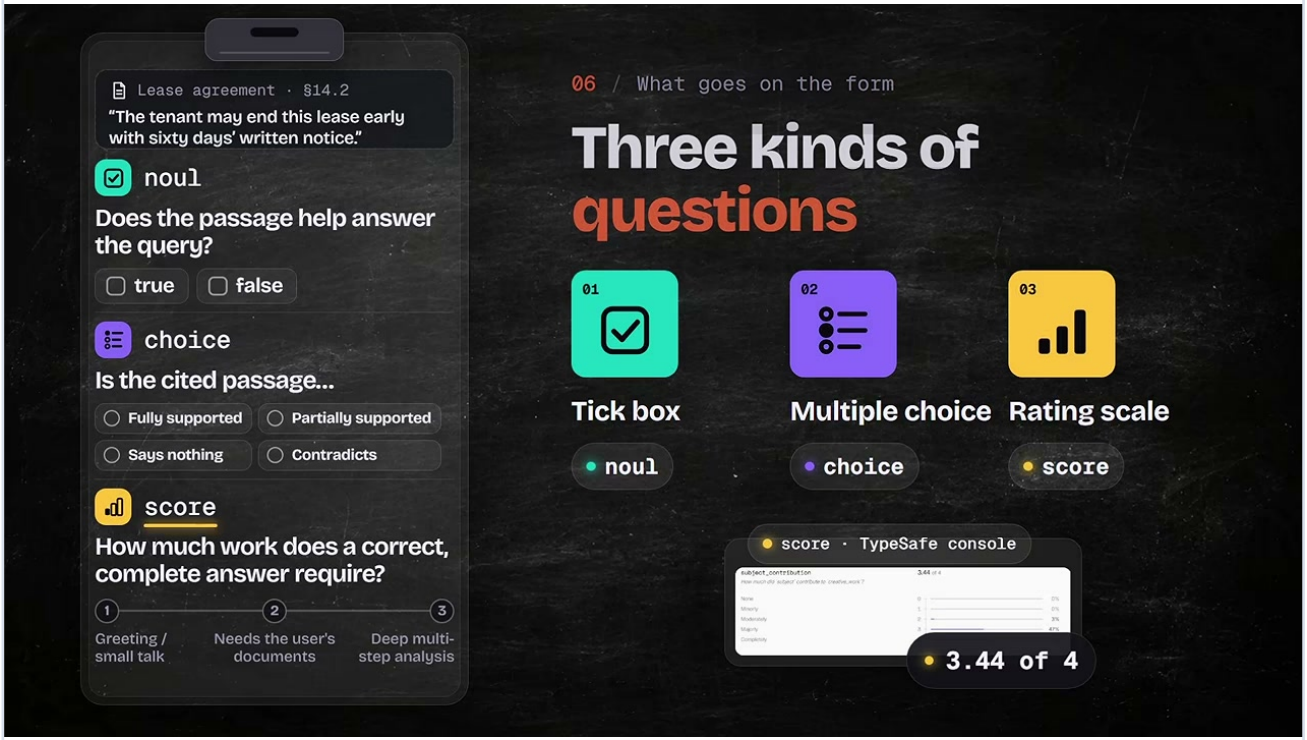
- One shown claim is labeled partially supported, while another is labeled fully supported.
- The model also returns a score that the developer can incorporate into custom application logic.
- No calibration evidence is shown, so any threshold should be treated as task-specific and validated before automation.

CORE IDEA

Scores enable escalation or filtering policies, but operational safety depends on measured error rates rather than the score label alone.

Three Question Types Form the Decision Schema

A single call can combine yes/no, multiple-choice, and rating-scale judgments.



NOTES

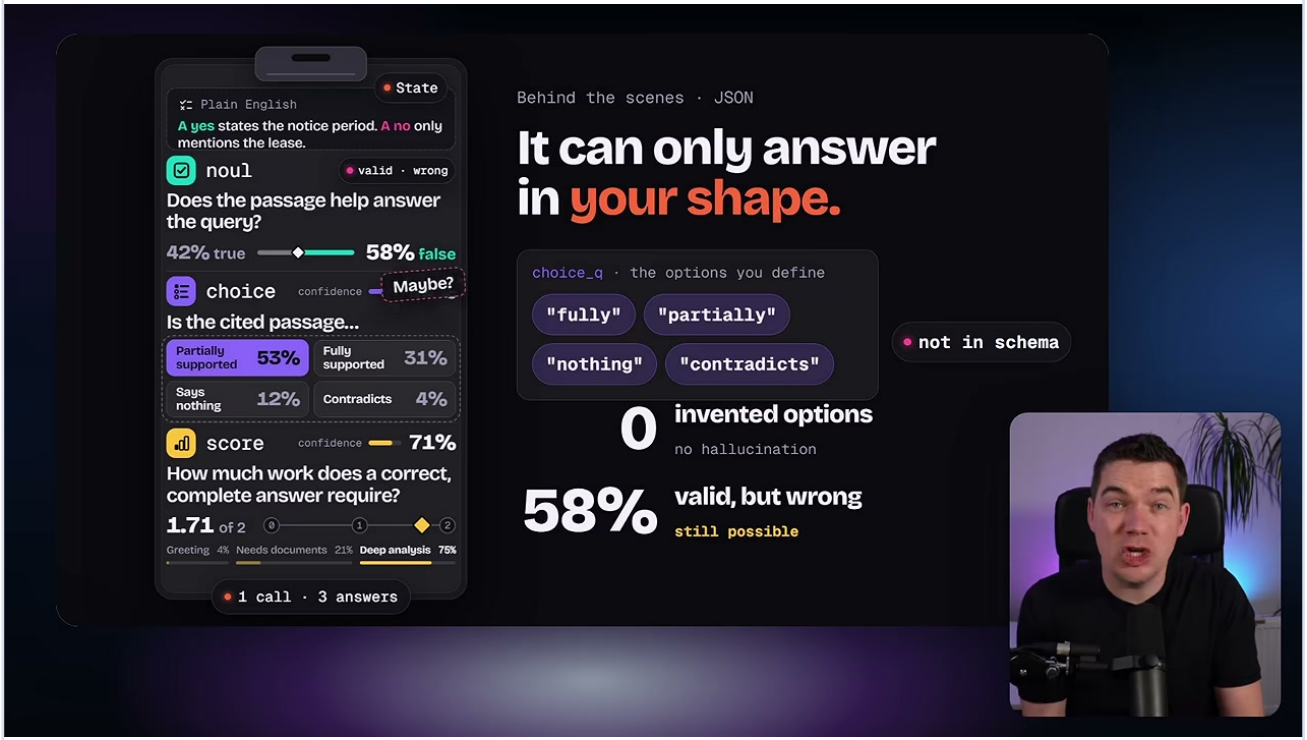
- A yes-or-no item acts like a checkbox for a binary judgment.
- A choice item selects one option from a developer-defined list.
- A score item places the state on a defined rating scale, and several items can be answered together.

CORE IDEA

These primitives let developers express several bounded judgments in one structured request instead of parsing free-form model text.

Schema Compliance Does Not Guarantee Correctness

The model is constrained to defined options, but it can still choose the wrong one.



NOTES

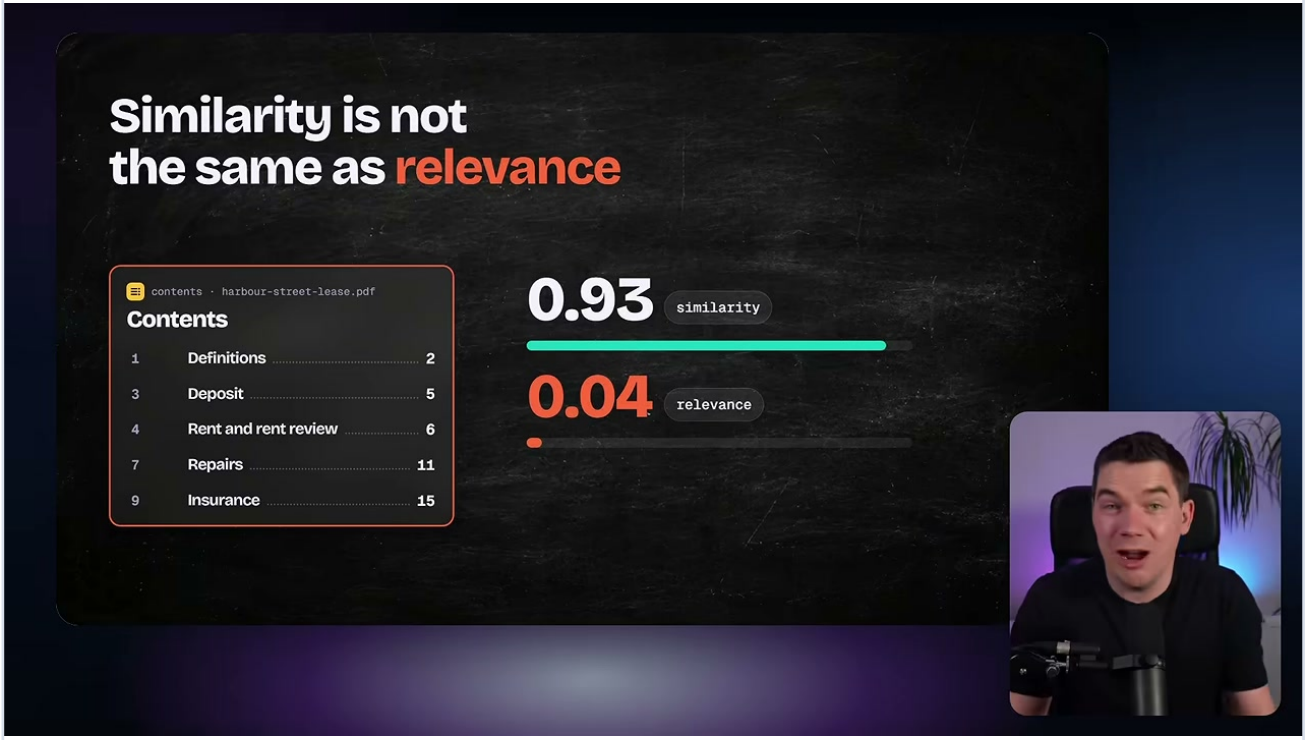
- The input state and question definitions are represented in JSON.
- The output is constrained to the shape and options defined by the developer.
- Schema locking avoids invented options, but it does not prevent an incorrect decision.

CORE IDEA

Structured output removes a parsing failure mode, not the need for evaluation, error handling, and human review where consequences are high.

Similarity Is Not Relevance

A search result can resemble the query without containing an answer.



NOTES

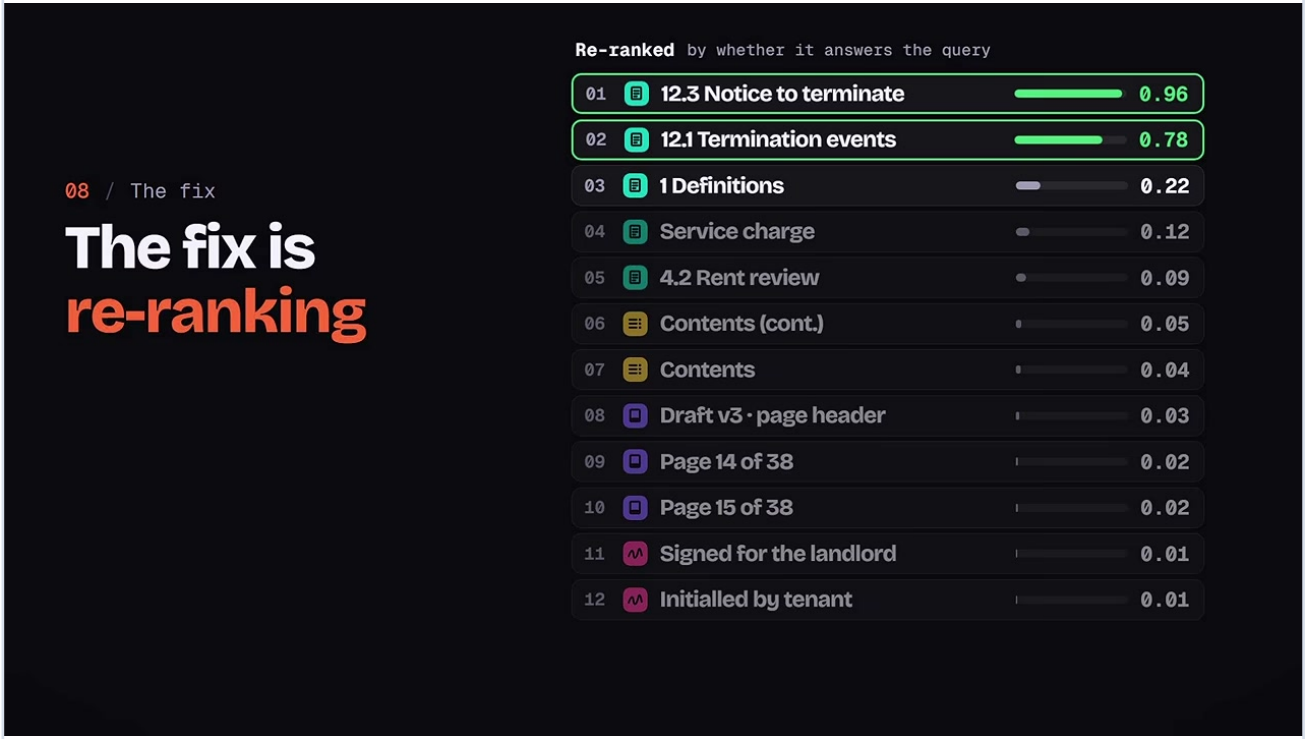
- Retrieval algorithms may return semantic, exact, or partial matches to the query.
- A table of contents can match many terms while answering none of the questions itself.
- Without reranking, useful and noisy chunks alike are loaded into the agent's context.

CORE IDEA

Treating similarity as sufficient evidence wastes context and can degrade the quality of downstream reasoning.

Reranking Reorders Results by Answer Utility

The second pass asks whether each candidate actually helps answer the query.



NOTES

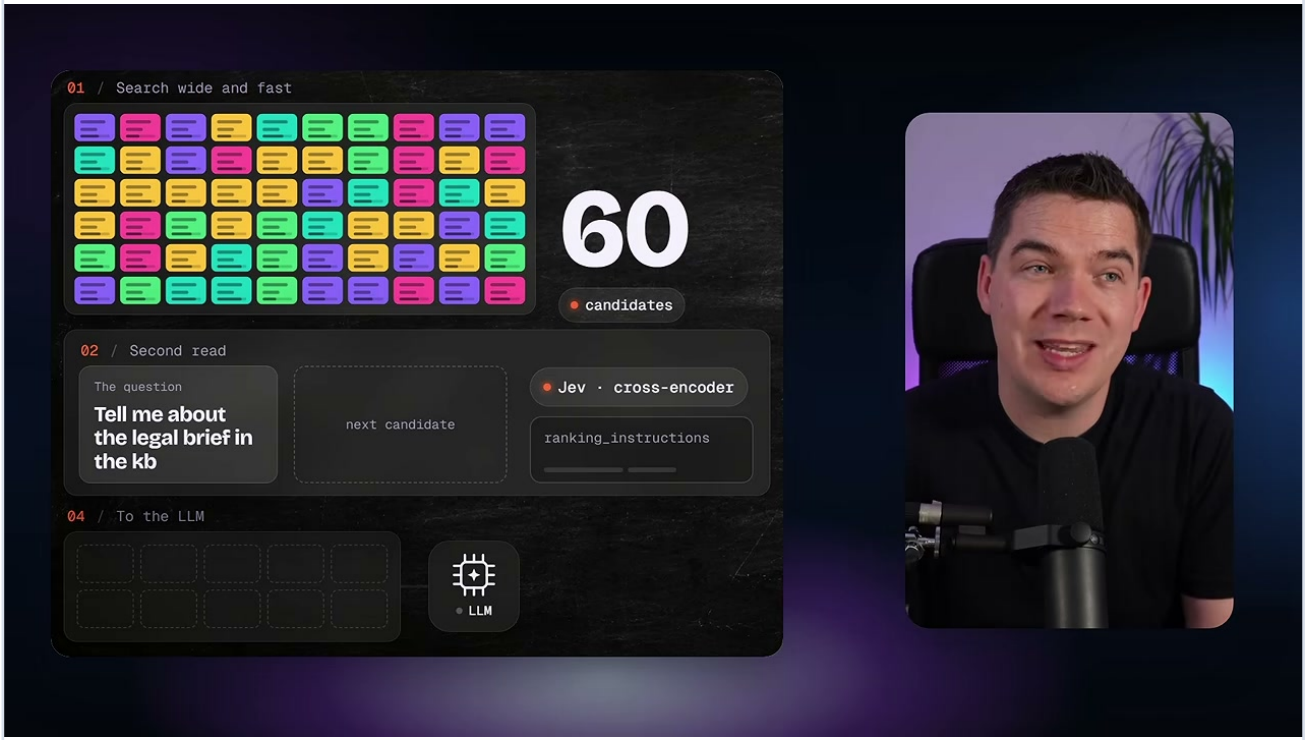
- The first retrieval stage searches broadly and quickly for possible documents or chunks.
- A second stage evaluates every candidate against the actual question.
- Results are reordered by whether they answer the query rather than by similarity alone.

CORE IDEA

The two-stage design preserves broad recall while improving the precision of what enters the model's limited context.

Search Wide, Then Read Candidates Again

The app retrieves 60 candidate chunks before the relevance pass.



NOTES

- Stage one builds a broad pool of possible chunks using fast retrieval.
- Stage two gives every candidate a closer read against the question that was asked.
- The pool of 60 is the demonstrated application setting, not a universal recommended candidate count.

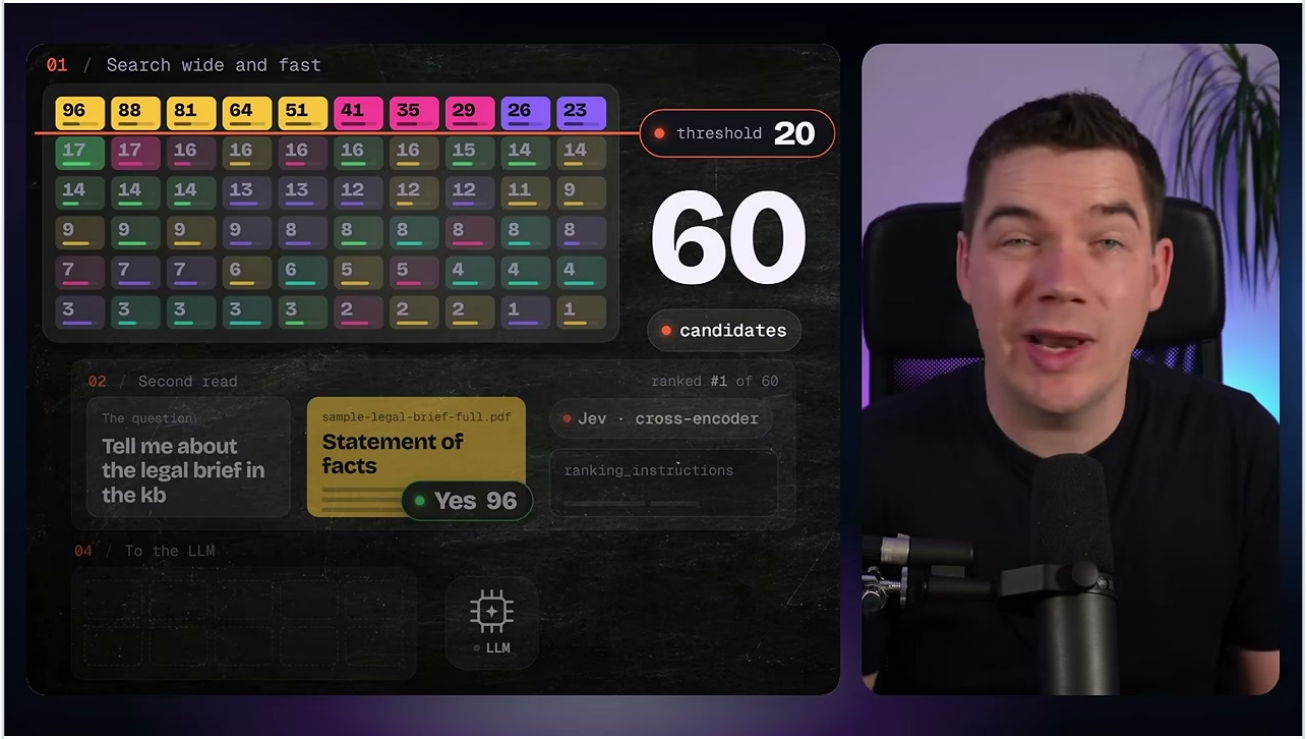
CORE IDEA

Candidate-pool size is an engineering trade-off between recall, reranking work, and downstream context volume.

Source: https://www.youtube.com/watch?v=ollve0D8pJQ

Thresholding Drops Low-Relevance Chunks

A sample policy removes results below 20%, potentially shrinking 60 candidates to 10.



NOTES

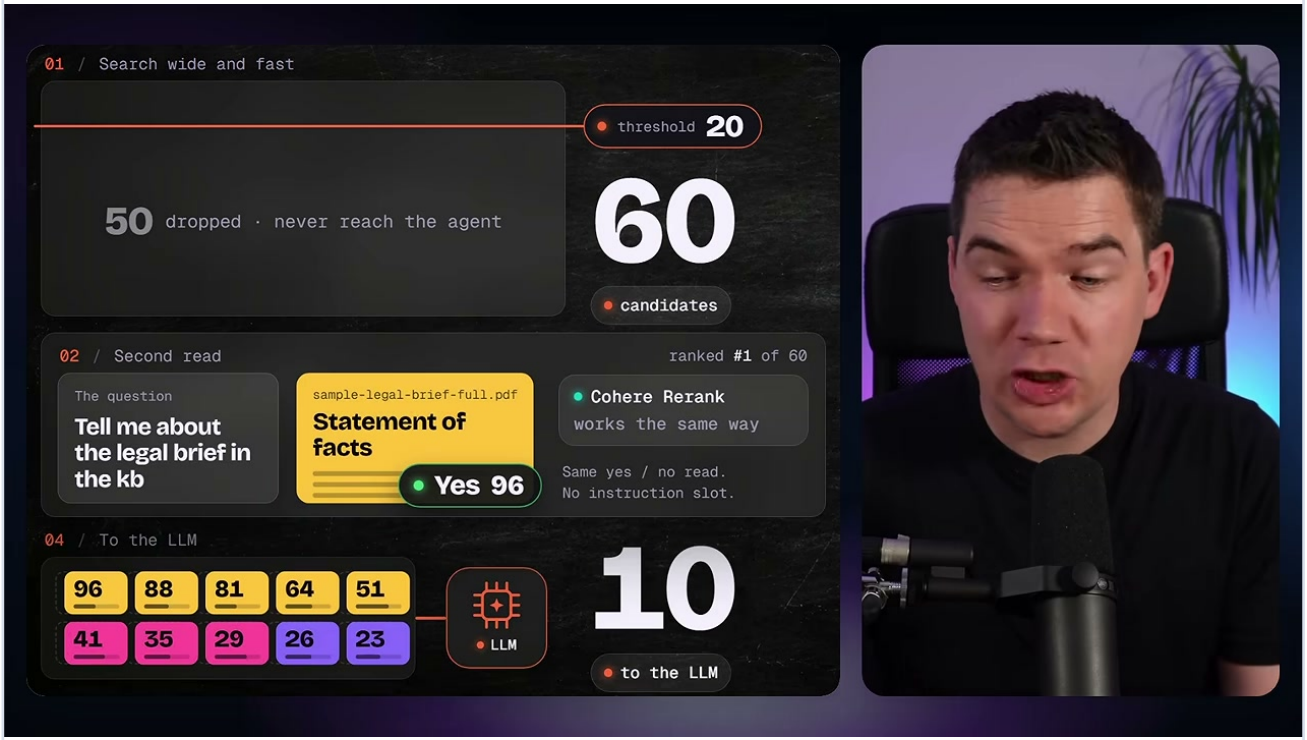
- A code-level threshold prevents low-scoring chunks from reaching the agent.
- The example uses a 20% cutoff and illustrates 60 retrieved candidates becoming about 10 LLM inputs.
- The threshold and counts are examples, and Jev's confidence values are not independently shown as calibrated.

CORE IDEA

Filtering can reduce noise and token use, but an aggressive unvalidated cutoff can also discard evidence the answer needs.

Steerable Reranking Separates Query from Criteria

Developer instructions can influence scoring independently of the search terms.



NOTES

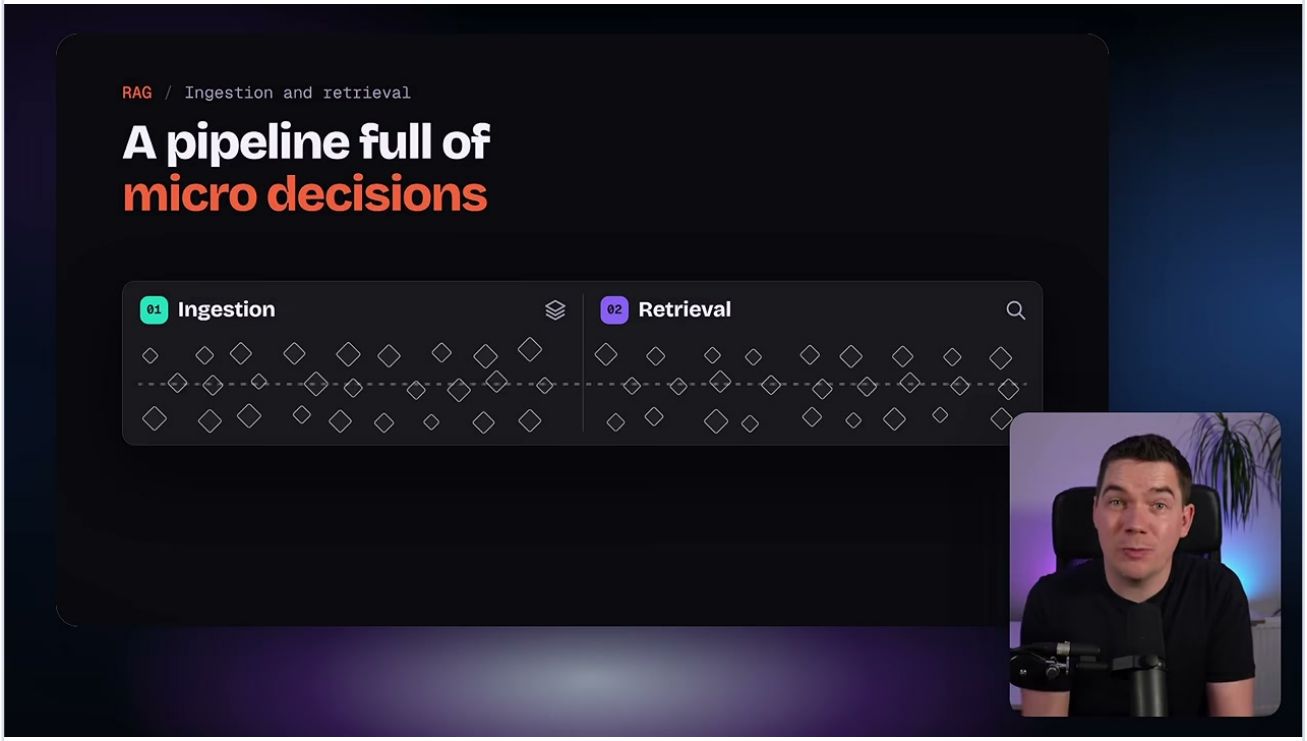
- The reranker receives all candidate chunks together with the original query.
- A separate instruction asks it to prioritize documents explicitly titled or described as a legal brief.
- Because the agent supplies the instruction, search terms and ranking criteria can differ.

CORE IDEA

Separating retrieval from ranking intent lets an agent express what makes a result useful even when those criteria are absent from the query.

Proposed Decision Points Across the RAG Pipeline

The demonstrated pattern is extended from retrieval into ingestion and indexing.



NOTES

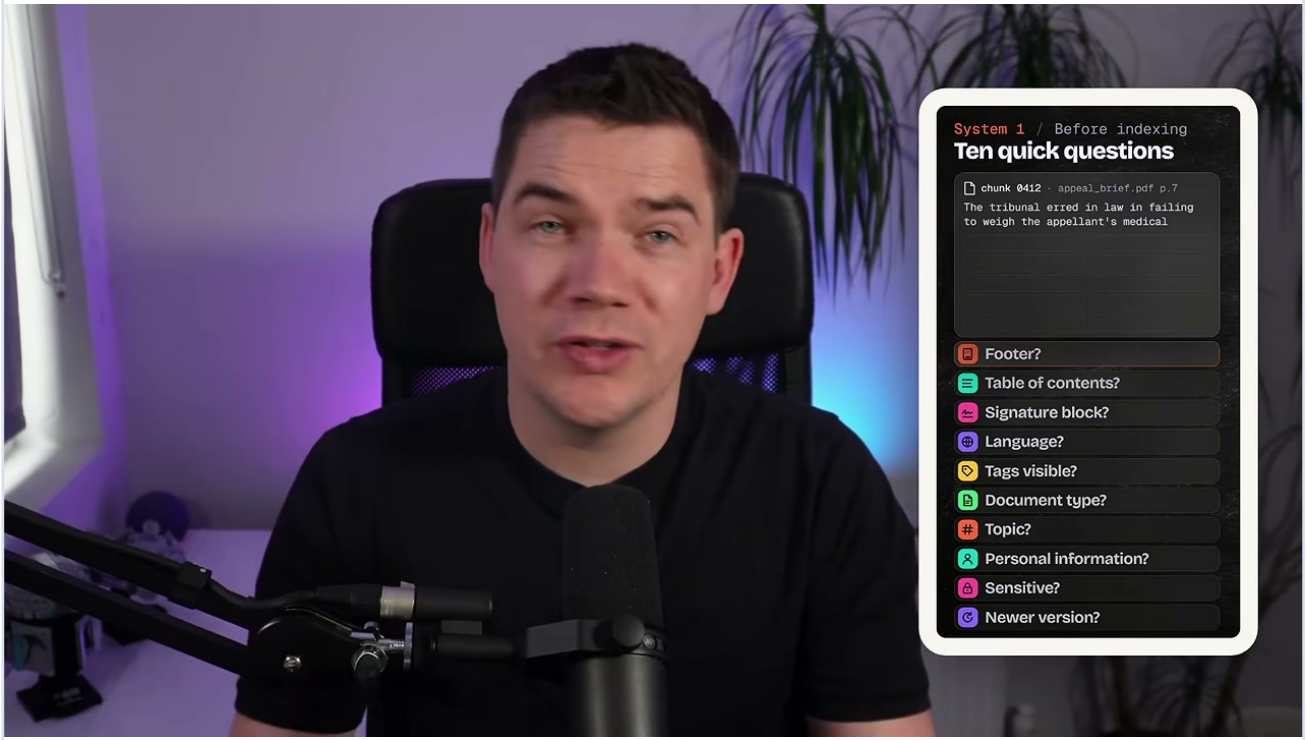
- The proposal treats ingestion and retrieval as pipelines containing many small classification decisions.
- A decision model could question each chunk before it is indexed and later evaluate retrieved evidence.
- These broader pipeline roles are proposed use cases, not demonstrated or benchmarked results.

CORE IDEA

Precomputing useful metadata could make later filtering more precise, but the value and error cost must be tested for each corpus.

Proposed Ingestion-Time Chunk Metadata

Each chunk could be labeled for structure, language, topic, sensitivity, and version status.



NOTES

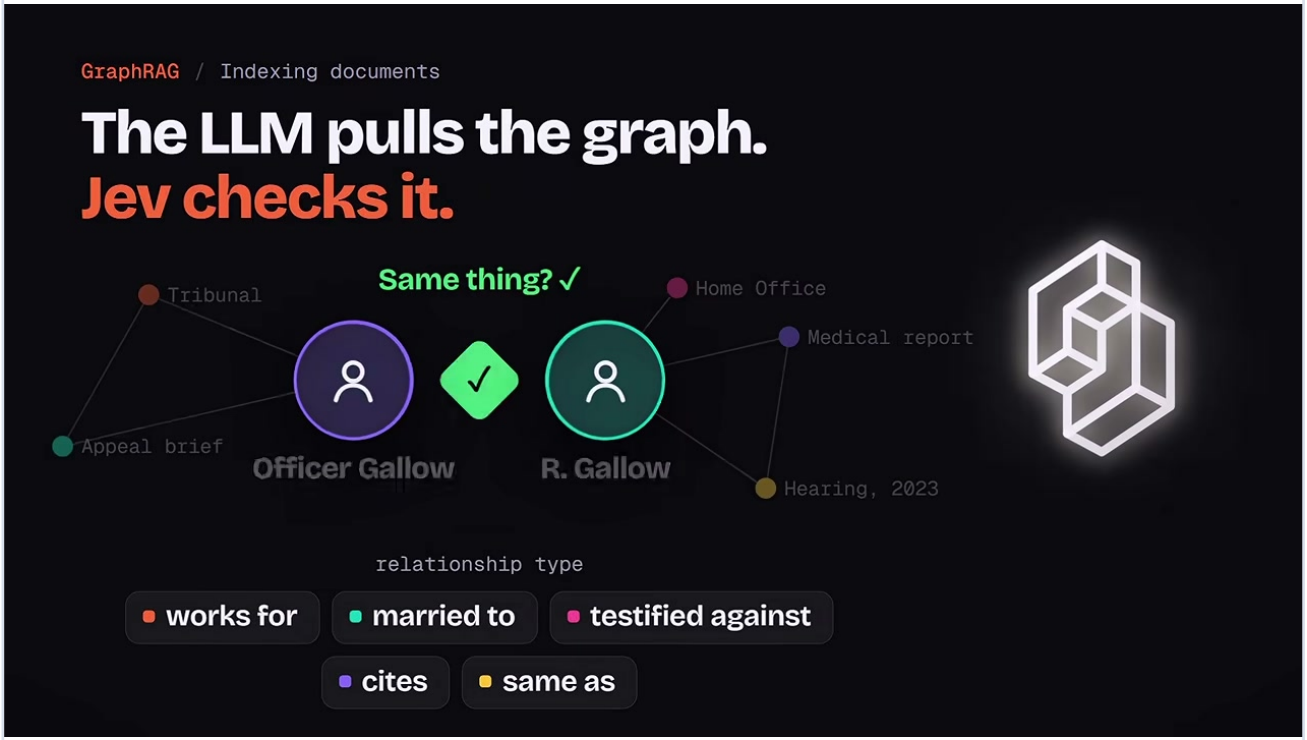
- Suggested labels include footer, table of contents, signature block, language, document type, and topic.
- Other proposed checks include personally identifiable information, sensitive content, and whether a newer version exists.
- These ingestion checks are proposals; their accuracy, latency, and operational benefit are not demonstrated here.

CORE IDEA

Reliable metadata could support later filters and supersession rules, but classification mistakes could hide or mishandle important content.

Proposed Graph-RAG Entity and Relation Checks

A decision model could judge entity equivalence, relation type, and passage support after extraction.



NOTES

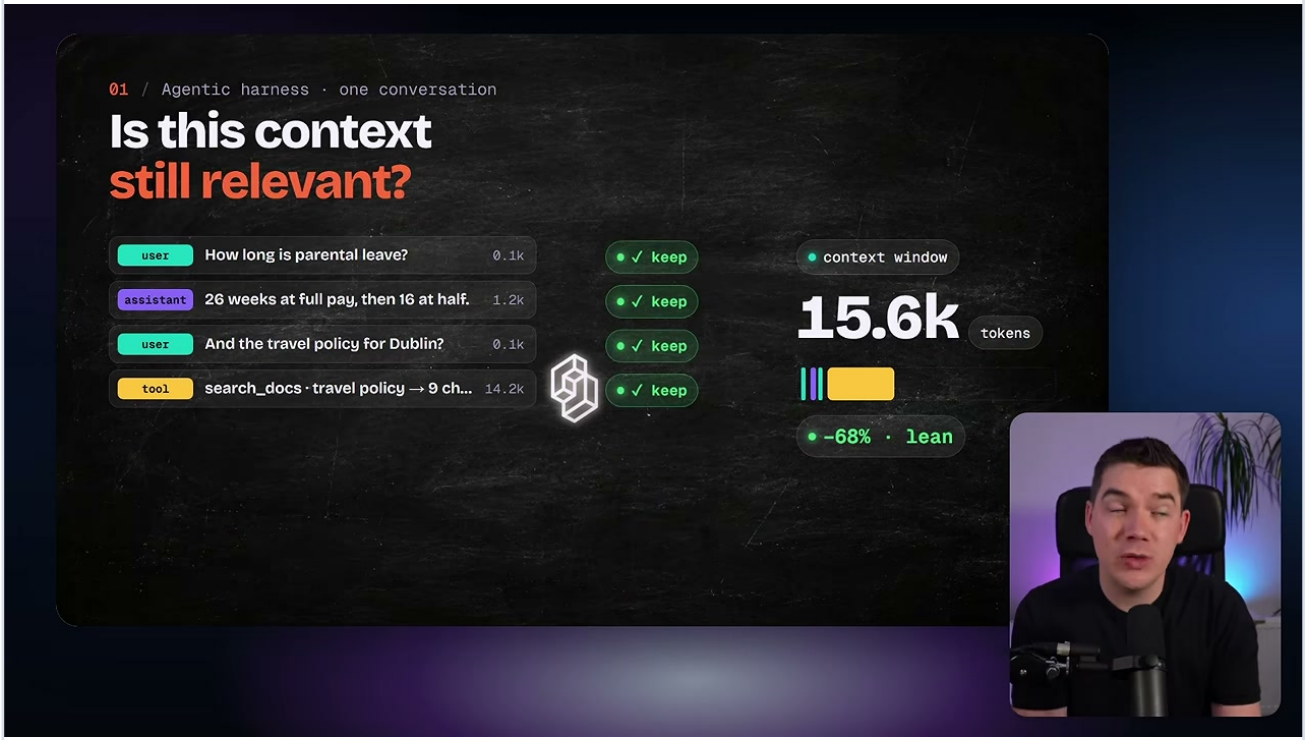
- An LLM would still extract entities and relationships from indexed documents.
- Jev could then decide whether two entities refer to the same thing and select a relation type from a defined list.
- Pairwise matching and support checks are proposed uses referenced from a cookbook, not independently benchmarked.

CORE IDEA

A bounded verification pass could improve graph consistency, but pairwise errors can propagate through the knowledge graph.

Proposed Turn-by-Turn Context Pruning

A decision model could ask whether earlier context remains relevant after every conversational turn.



NOTES

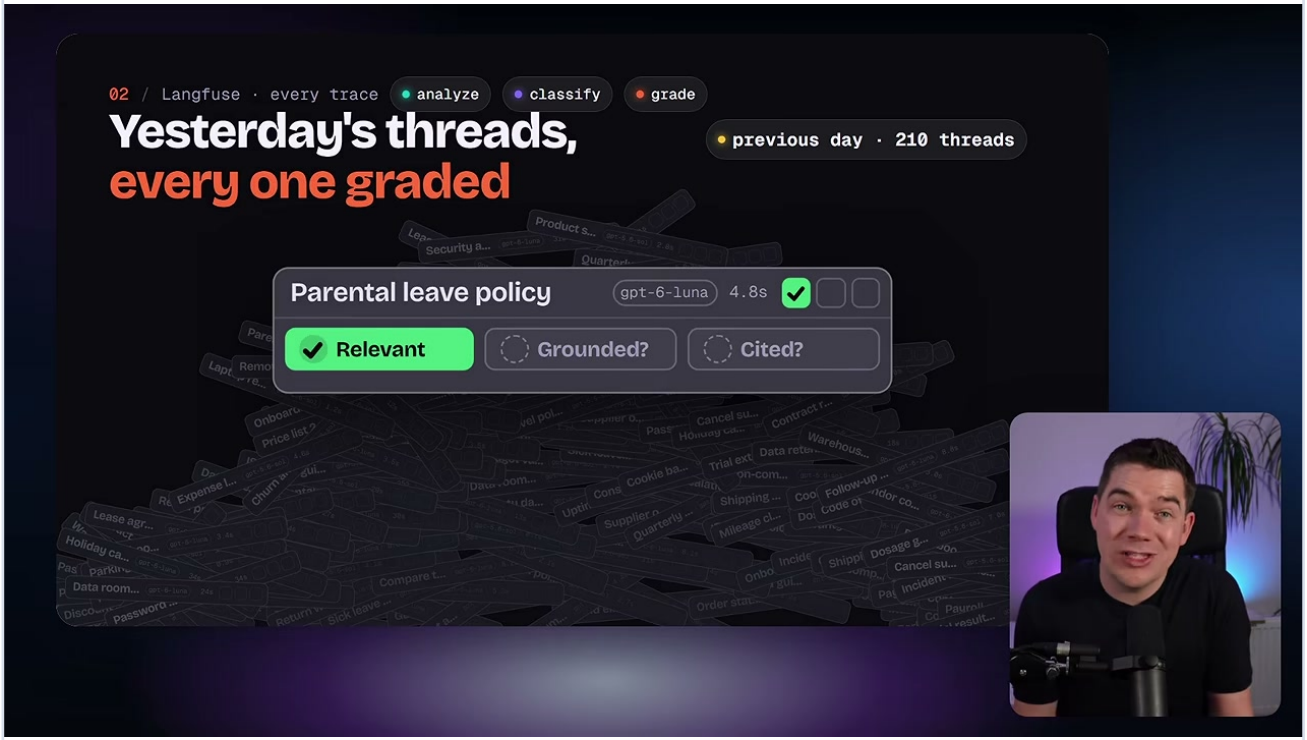
- One proposed use evaluates context relevance after every conversation turn.
- Context judged no longer relevant could be dropped to keep the working context lean.
- This is a proposed use case; no retention accuracy, quality impact, latency, or cost benchmark is shown.

CORE IDEA

Context pruning may control token growth, but false removals can erase facts or constraints needed later in the conversation.

Proposed Batch Evaluation of Agent Traces

Yesterday's conversations could be classified for relevance, grounding, and citation behavior.



NOTES

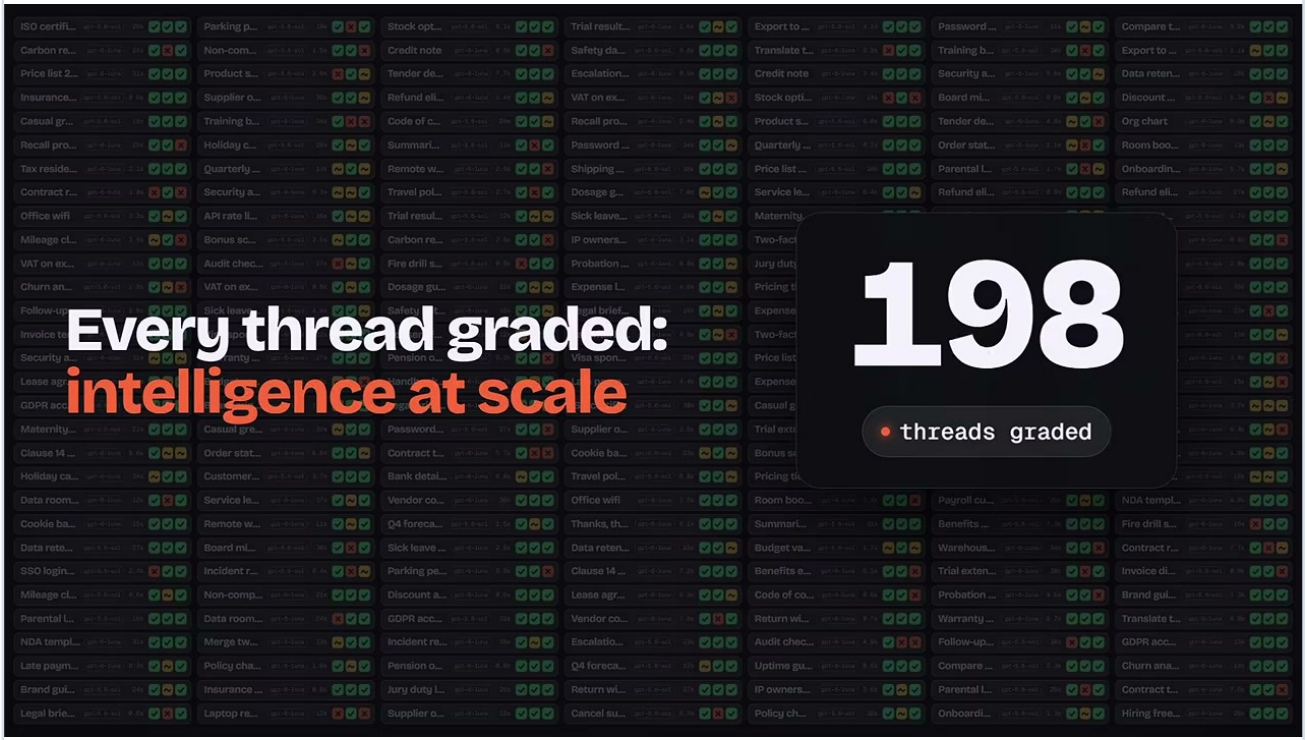
- Langfuse traces could provide the conversations and answers for offline review.
- Suggested checks ask whether retrieved information was relevant, the answer was grounded, and sources were cited.
- Evaluating every thread instead of a sample is a proposal; no quality, throughput, latency, or cost benchmark is supplied.

CORE IDEA

Automated review could expand observability coverage, but its labels need comparison with trusted human judgments before they become quality metrics.

Decision-Making at Scale Is the Closing Proposal

Low-cost bounded judgments could evaluate all conversations rather than a small sample.



NOTES

- Manual or LLM-based review has often relied on sampling because the trace volume is large.
- The proposed alternative is to classify all conversations with a specialized decision model.
- This scale claim is prospective and based on the demo; no end-to-end benchmark, calibration study, or production result is provided.

CORE IDEA

Full-coverage evaluation could reveal rare failures that samples miss, provided the evaluator's own error rate and operating cost are measured.

Source: https://www.youtube.com/watch?v=ollve0D8pJQ